Network-Aware Predictive Digital Twin Framework for Resilient and Self-Reconfigurable Cyber-Physical Production Systems

Abstract. Delayed observations and missing sensor transitions complicate supervisory decisions in conveyor-based production cells. This paper presents a simulation-stage digital twin framework that separates process simulation, sensing, communication, estimation, sequence control, prediction, and independent evaluation. Software reconfiguration is restricted to a cycle-latched speed limit and a hold request; it neither changes the mechanical layout nor automatically resumes an interrupted workpiece. A command-history estimator propagates workpiece position between timestamped optical observations. A reduced predictor evaluates 54 stopping trajectories over a 0.5-second horizon, while an independent MuJoCo belt-and-workpiece predictor provides a more detailed alternative. A matched study comprises 24 simulated runs covering nominal operation, missing pickup detection, communication delay, packet loss, and command-to-motion mismatch. Eighteen runs complete placement and six terminate in a supported hold, without violating the stated terminal criteria. In the two delayed-observation cases, enabling reconfiguration permits completion whereas disabling it results in a hold. A separate Docker timing exercise finds that the reduced predictor completes all 15 measured calls within a configured 20-millisecond allowance. The MuJoCo predictor completes none within that allowance and remains unsuitable for that deadline in its current implementation. These observations support traceable software-level experimentation, rather than measured fault-diagnosis accuracy or verified physical resilience. Native grouping and configurable workers reduce complete-forecast time in checked cases but do not meet the 20-millisecond target. Physical calibration, live controller integration, and hardware validation remain unreported.

Keywords: Digital twin · Network-aware supervision · State estimation · Software reconfiguration · Virtual commissioning

1 Introduction

A geometric representation alone does not establish that a digital twin can support production decisions. Kritzinger et al. [4] distinguish digital models, shadows, and twins according to data integration. For a moving workpiece, a decision additionally depends on when an observation was generated and what motion

occurred before its reception. Communication delay and packet loss are established concerns in networked control [3]; the age of timestamped information provides a representation of freshness [6]. Here, digital twin framework denotes the intended architecture. Its reported implementation is a simulation testbed, not an already synchronized physical twin.

1.1 Related Research and Remaining Development Needs

Ademujimi and Prabhu [1] combined equipment and discrete-event simulation to generate interventions for learning Bayesian-network structures for manufacturing fault diagnostics. This provides a precedent for exploiting virtual faults when physical fault data are limited. The present objective is different: an unresolved observation must lead to a continue-or-hold decision while a workpiece is moving. A unique physical fault cause cannot simply be assumed from one missing binary transition.

Fu et al. [2] developed a digital-twin-based framework that monitors disturbances, optimizes manufacturing-line configurations, and evaluates alternatives through generated simulation. Their work addresses line-level configuration and disturbance handling. We instead fix the equipment arrangement and investigate software actions: limiting conveyor speed or requesting a hold. The resulting evaluation must distinguish continued production from interruption and must not count a supported hold as a completed product.

MuJoCo supplies multibody and contact dynamics for model-based control [5]. Engine selection alone does not establish calibrated pneumatics or guarantee that an online forecast finishes before the next decision. We therefore separate plant simulation from two supervisory prediction alternatives and measure computation time explicitly. The focused research question is whether timestamped observations and bounded motion forecasts support traceable decisions under controlled disturbances without exposing simulator ground truth to the supervisor. This synthesis of selected relevant studies is not a systematic review or evidence that the complete literature lacks such integration.

1.2 Scope and Contributions

The implemented contributions are an observation-constrained modular architecture; a command-history estimator and continue-or-hold supervisor; a cycle-latched speed-degradation policy; and reproducible virtual experiments with separate timing measurements. Resilience is examined through terminal support and completion in specified fixtures, not through a general resilience index. Self-reconfiguration refers to the software speed limit and hold, not mechanical rearrangement, sensor replacement, or autonomous recovery. Physical-validation sections remain intentionally unfilled.

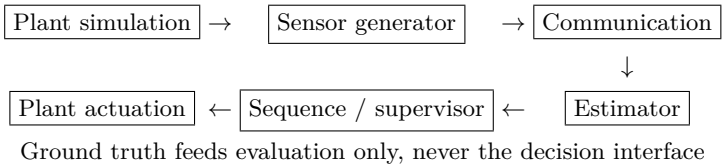


Fig. 1. Virtual information paths. Communication delivers observations to the estimator; true state is excluded from the decision interface.

2 System and Information Boundaries

2.1 Cell and Hybrid Representation

The cell comprises two conveyors, a transfer pusher, three photoelectric-sensor roles, an XY gantry, a vertical actuator, and a vacuum gripper. It handles one workpiece per reset. Nominal workpiece dimensions are $60 \times 60 \times 60$ mm in the parameterized profile; these are preliminary user-reported dimensions, not a physical measurement campaign reported here. CAD V5 and the functional MuJoCo scene are not claimed to be fully synchronized. Mass, friction, suction-related quantities, and motion response remain engineering assumptions awaiting identification.

The hybrid architecture can be written as

$$x_{k+1} = F_\theta(x_k, u_k, q_k), \qquad q_{k+1} = G(q_k, z_{\le k}, \widehat{x}_k), \tag{1}$$

where x_k denotes continuous simulated states, q_k the sequence phase, u_k actuator requests, and θ the parameter profile. Equation (1) describes the software architecture, not a newly identified physical state equation. The sequence covers admission, pushing, transport, alignment, pickup, transfer, release, return, completion, and latched hold. Endpoint switches support cylinder transitions. Independently observed feedback for XY completion, intermediate vertical positioning, and vacuum confirmation is not fully available in the evaluated configuration.

2.2 Observation-Constrained Decisions

Figure 1 separates the decision path from evaluation. Simulated state supplies the sensor generator and an independent evaluator. The estimator receives only observations delivered through the communication model. Missing feedback remains unavailable rather than silently becoming a negative sensor reading. Messages carry session identity, reset generation, and sequence number; wrong-session, old-generation, and repeated messages are rejected. These local checks are not an authenticated industrial communication protocol.

3 Estimation and Predictive Supervision

3.1 Information Age and Motion Estimation

Let an accepted observation have source time τ_i and reception time r_i. At decision time t, its information age is

$$a(t) = t - \tau_{\ell(t)}, \qquad r_{\ell(t)} \le t, \tag{2}$$

where $\ell(t)$ indexes the newest accepted observation. Age concerns generation time, not simply time since reception [6]. Evaluation uses a shared simulation clock. Real PLC clock offset and synchronization uncertainty have not been measured.

Between optical edges, the estimator integrates issued belt commands instead of reading simulator velocity. Let $D_i = \int_{\tau_{i-1}}^{\tau_i} u_b(t)\,dt$, V_i be command-speed total variation, and N_i count commanded stops. The implemented propagation is

$$\widehat{y_i^-} = \widehat{y}_{i-1} + \widehat{g} D_i + N_i c_s, \tag{3}$$

$$b_i^- = b_{i-1} + \rho(\tau_i - \tau_{i-1}) + \tau_u V_i + N_i c_s. \tag{4}$$

Here $\widehat{g}$ is an estimated command-to-motion gain, c_s a stopping-coast allowance, ρ drift allowance, and τ_u command-response uncertainty. The interval half-width b_i is an engineering allowance, not a calibrated confidence interval.

An optical rising edge refers to a leading face, not the center. With beam coordinate p_j, workpiece length L, and sample-window correction $h_i = \widehat{g} D_i/2$, the history-enabled estimator resets position and allowance as

$$\widehat{y}_i = p_j - L/2 + h_i, \quad b_i = b_0 + \max\{h_i, g_{\max} D_i + N_i c_s - h_i\}. \tag{5}$$

Gain is updated from commanded travel between belt-sensor edges, subject to plausibility checks. Commanded motion after the observation source time is accounted for separately when deciding whether pickup uncertainty is acceptable. Admission centering still depends on provisional geometry and commanded feed motion; arbitrary arrivals are outside the evaluated scope.

3.2 Finite-Scenario Continue-or-Hold Decision

The reduced predictor uses estimated position $\widehat{y}$, velocity $\widehat{v}$, uncertainty b, and observation age a. For uncertainty offset ϵ, velocity offset ν, reaction delay δ, and action $\ell \in \{0, 1\}$, define $v_\nu = \max(0, \widehat{v} + \nu)$ and $b_a = b + \rho a$. The stopping surrogate is

$$\begin{aligned} y_{\ell,\epsilon,\nu,\delta}(s) = \widehat{y} + \epsilon b_a + v_\nu a + v_\nu \min(s, \delta + \ell T) \\ + c_s \operatorname{clip}\left(\frac{s - \delta - \ell T}{h}, 0, 1\right), \end{aligned} \tag{6}$$

where T is the control interval and h the reduced prediction grid. In history-enabled operation, the supplied forecast speed is an upper bound derived from

issued speed and allowed gain, rather than an independently measured belt velocity. Action $\ell = 0$ requests hold now; $\ell = 1$ allows one additional interval before stopping. The coast term is not identified braking physics. For forward limit $y_{\text{lim}} = y_{\text{end}} - L/2 - d_{\text{reserve}}$, the action margin is

$$m_\ell = \min_{\epsilon,\nu,\delta,j} \{y_{\text{lim}} - y_{\ell,\epsilon,\nu,\delta}(jh)\}. \tag{7}$$

Continuation requires a complete forecast, $m_1 \geq 0$, and completion within the computation allowance. Otherwise the supervisor requests hold. If $m_0 < 0$, holding is not thereby proved safe; the rule still rejects additional continuation. This is a supervisory screen, not an optimal-control or stability theorem.

3.3 Contact Forecast and Software Reconfiguration

The alternative predictor constructs an independent MuJoCo belt/workpiece scene from estimates and the profile, without cloning live plant state. It evaluates belt continuation and stopping and checks forward, lateral, and vertical margins. It excludes the full gantry, pressure dynamics, and calibrated suction failure. Its fixed-footprint checks are not a general rotated-body collision certificate. The two predictors thus have different model detail and numerical workloads.

Repeated age warnings or observation gaps can latch NORMAL into DEGRADED. In transport, the fixed policy issues

$$u_b^{\text{issued}} = \min\{u_b^{\text{requested}}, u_{\text{cap}}\}. \tag{8}$$

Unusable observations, contradictory endpoint switches, missing expected transitions, or supervisory violations can request HOLD. Belt requests then become zero while the existing vacuum request is retained; this does not confirm physical vacuum. Communication recovery does not clear the hold automatically. The implemented reset handshake prepares a fresh simulation cycle through distinct reset, acknowledgement, and start steps. It does not resume a previous suspended workpiece.

4 Virtual Evaluation Methods

4.1 Settings and Timing Measurements

The profile uses a 2 ms physics step and a 20 ms control interval. Both predictors evaluate 54 trajectories: two actions, three position offsets, three velocity offsets, and three reaction delays. The horizon is 0.5 s. The reduced grid is 20 ms, while contact integration uses 2 ms; equal scenario counts do not imply equal work. Nominal and degraded transport commands are 0.16 and 0.12 m/s. They are selected simulation settings, not measured physical capacities.

Three synthetic estimated states are used for timing. Each receives one warm-up call and five measured calls per predictor and allowance, yielding 15 measured calls per group. Initialization is recorded separately and warm-up samples are

retained outside the summaries. External elapsed time includes the invocation wrapper and result creation. We report medians, nearest-rank empirical p95, completion counts, and observed deadline exceedances. The configured allowance is 20 ms; a separate 1 s functional allowance permits inspection of complete forecasts without replacing the configured deadline.

The recorded Docker environment contains Python 3.12.16, MuJoCo 3.16.0, and NumPy 2.5.4 on WSL2 Linux. Processor identity and controlled background load were not recorded for this campaign. Timing is therefore descriptive, not a worst-case execution-time bound or a cross-platform benchmark.

4.2 Matched Disturbances and Evaluation Criteria

Six scenarios vary one factor: nominal operation; pickup-sensor stuck low; 20 ms message delay; 5% packet loss; and plant belt speed at 0.85 or 1.15 times its command. True plant-speed changes are not simultaneously supplied to the estimator as known gains. Each scenario uses two network seeds and two policy conditions, giving 24 runs. The conditions differ in whether the fixed-policy reconfiguration module, comprising the cycle-latched cap and health-triggered holds, is enabled. This is not an isolated speed-cap ablation. Both retain estimation and predictive supervision: this is not a sequential-PLC-versus-DT comparison. Changing seeds does not make deterministic scenarios independent replicates.

Full-cycle trials explicitly enable open-loop assumptions for unconfirmed XY, intermediate vertical positioning, and vacuum feedback. Strict operation instead holds before these steps. Results under these assumptions do not demonstrate physical feedback control. A terminal criterion is met by evaluator-confirmed placement after completion or evaluator-confirmed support during hold. Speed-cap adherence and policy logging are also checked. Ground truth is restricted to evaluation. This study uses the 1 s functional prediction allowance, so terminal outcomes do not establish 20 ms operation. No threshold tuning is performed within the campaign.

5 Virtual Results

5.1 Forecast Computation

Table 1 is generated from the recorded 2 November Docker evidence, before the implementation optimization reported below. The reduced predictor completes all 15 measured calls under the configured allowance, with p95 3.54 ms. The contact predictor completes none of its 15 calls within that allowance. With the functional allowance, it completes all 15 with p95 596.29 ms. Its incomplete 20 ms calls request hold; returning hold after an observed overrun is not a hard-deadline guarantee. The current contact predictor is not demonstrated to support a 20 ms supervisor on this host. The result does not show that the reduced model is physically more accurate.

Table 1. Observed Docker forecast timing; 15 measured calls per row after warm-up

Model	Allowance (ms)	Median (ms)	p95 (ms)	Complete /15	Overruns /15
Reduced 1D	20	2.76	3.54	15	0
Reduced 1D	1000	2.30	3.02	15	0
MuJoCo contact	20	20.48	21.09	0	15
MuJoCo contact	1000	460.37	596.29	15	0

Table 2. Matched reconfiguration-module study: completion (D) and supported hold (H)

Condition	Off: D	Off: H	On: D	On: H
Nominal	2	0	2	0
Pickup sensor stuck low	0	2	0	2
Delay: 20 ms	0	2	2	0
Loss: 5%	2	0	2	0
Plant speed: 0.85 command	2	0	2	0
Plant speed: 1.15 command	2	0	2	0

5.2 Disturbance Outcomes

All 24 runs meet the terminal, cap, and logging checks: 18 complete placement and 6 end in supported holds. Table 2 preserves this distinction. In the two delayed-observation cases, enabling reconfiguration permits completion; disabling it produces holds for excessive pickup uncertainty. Both conditions hold when pickup detection is absent. These are fixture-level observations, not a diagnosis sensitivity, false-alarm rate, or throughput gain.

The earlier artifact also records 97 logic checks and end-to-end integration checks for strict/open-loop operation, replay, prediction, and deadline handling. Check counts support implementation consistency; they are not performance sample sizes. Source and configuration provenance are identified in [7]. No physical result is inferred from passing these checks.

5.3 Native Rollout Optimization

A subsequent implementation replaces repeated Python stepping with the installed native rollout facility [8]. It groups candidates by their stopping-step index, evaluates running prefixes, and starts the stopped-model segment with the same time, position, and velocity transfer as the reference implementation. Every returned step, including the initial state, contributes to the margin check. Physics parameters, the integration grid, and the candidate set are retained. Worker count is a separate software setting; configurations with one, four, and eight workers are evaluated rather than silently selected as a new default.

Twelve numerical fixtures, including zero speed, aged estimates, end-of-belt conditions, and off-grid stopping times, are checked at each worker count against

Table 3. Native rollout observations on 3 November: full-call median and p95 use the 1 s allowance; each count is out of 15 measured calls

Implementation	Median (ms)	p95 (ms)	Complete at 20 ms	Complete at 1 s
Reference loop	398.45	447.00	0	15
Native: 1 worker	137.37	151.62	0	15
Native: 4 workers	56.39	62.52	0	15
Native: 8 workers	50.29	54.65	0	15

the retained reference. Actions, scenario counts, integration-step counts, and minimum margins satisfy the declared checks; the observed maximum margin difference is zero with a comparison tolerance of 10^{-9} m. This tolerance tests two implementations of the same model and is not a statement about physical accuracy. A separate trajectory regression compares individual native states with successive reference steps.

Table 3 reports a separate 3 November Docker timing campaign, with 15 measured calls per implementation and allowance after warm-up. The observations show lower complete-forecast times for native grouping and additional workers. Calls are evaluated in sequence on the same host, without randomized ordering or controlled background load; no universal speedup or statistical significance is claimed. No variant completes a measured call within 20 ms. Native groups cannot be interrupted mid-call, and returning HOLD after a detected overrun can take longer than in the reference loop. The optional eight-worker end-to-end simulation also holds for a prediction-deadline overrun with evaluator-confirmed support. It is not counted as a completed product. Thus the optimization improves the observed full-forecast computation but does not establish a real-time physical supervisor.

5.4 Observation-Only Background Forecasts

To examine separation of computation from supervision, a spawned process runs the contact predictor while the reduced predictor retains command authority. A bounded channel permits one outstanding request and skips new submissions while busy. No controller, actuator writer, or live plant state is supplied to the worker. Each request binds a session, generation, profile digest, source sequence and time, estimate snapshot, and command revision. Returned results are screened for matching provenance, age, operating state, and complete finite scenario output. Simulation-time age and parent-process elapsed age are evaluated separately; this is not physical clock synchronization.

These checks admit observations only. In particular, a continuation forecast computed for an earlier one-interval action does not authorize continuation from the present state. Accepted and rejected replies are logged after the foreground command gate and cannot change a command or clear a hold. Worker initializa-

Table 4. Paced observation-only Docker cycles on 3 November; foreground runtime-call timing, not end-to-end latency

Condition	Steps	Median (ms)	p95 (ms)	Maximum (ms)	Over 20 ms (steps)
Disabled	697	0.32	2.59	4.92	0
Observation only	697	0.47	3.51	12.63	0
Incomplete worker	697	0.45	3.01	4.57	0

Observation-only cycle: 35 requests, 34 admitted observations, 1 discarded.
Incomplete-worker cycle: 81 incomplete forecasts rejected.

tion and shutdown occur outside the decision step; a response timeout disables observation processing without restarting the worker within that step.

Three paced virtual cycles compare background disabled, background enabled, and an intentionally incomplete background worker. The foreground allowance remains 20 ms. Each uses the same plant and open-loop feedback assumptions. Logged simulation times, phases, estimates, and commands are compared exactly, excluding session identifiers and measured computation durations. A separate probe terminates the worker and checks that the foreground call still returns. This probe tests containment of that software failure, not continued physical production after arbitrary failures.

Table 4 describes the recorded Docker run. All three cycles finish placement as confirmed by the evaluator and have identical compared traces. Complete background observations are received, incomplete results are discarded, and worker processes are closed. The probe that terminates the worker also passes. Timing covers the foreground runtime call including channel polling, but excludes plant integration, file output, pacing, initialization, and shutdown. Each condition contains one cycle; repeated decision steps are not independent production trials. These measurements do not qualify complete system deadlines or show that the contact forecast itself finishes within 20 ms.

6 Physical Validation

Reserved: no physical measurements or validation results are reported.

6.1 Physical Setup and Calibration

6.2 Live PLC Synchronization and Network Measurements

6.3 Physical Fault Trials and Placement Accuracy

6.4 Comparison with Sequential PLC Operation

7 Discussion and Limitations

The evidence supports an inspectable software testbed, not a closed-loop physical twin. A supported hold is a virtual terminal condition, not demonstrated process availability or guaranteed safety. A speed cap can preserve completion in a delayed-data fixture without improving nominal cycle time. Selecting only completions would conceal interruption costs; the tables retain all planned runs.

The principal computation limitation is the gap between complete contact forecasts and the configured allowance. Optimization must preserve numerical behavior while being evaluated against actual elapsed time; increasing the allowance alone changes the question. Selected states and repeated host calls do not characterize worst-case scheduling. The engineering interval and stopping-coast assumptions require physical identification before quantified risk statements are justified.

Sparse binary sensing leaves sensor, communication, and model errors partly ambiguous. Findings indicate suspicions rather than uniquely isolated physical failure causes. CAD-to-simulation consistency, sensor offsets, belt slip, vacuum holding, pneumatics, and axis feedback remain unvalidated. Simultaneous products and continuing an interrupted old workpiece are outside the demonstrated capability. Section 6 is left blank rather than substituting synthetic measurements.

8 Conclusion

A modular network-aware supervisory framework has been implemented and examined virtually. The matched campaign demonstrates completion or supported interruption in its specified fixtures and makes information and computation boundaries explicit. The reduced predictor meets its allowance in the measured calls; the current independent MuJoCo predictor does not. Physical calibration, live integration, and a sequential-PLC comparison remain unreported. These limitations delimit further work, rather than supporting claims of verified physical resilience or general performance superiority.

Declarations Awaiting Author Completion

Authors and affiliations: _____

Funding and competing interests: _____

Author-approved AI-use disclosure: _____

References

1. Ademujimi, T., Prabhu, V.: Digital twin for training Bayesian networks for fault diagnostics of manufacturing systems. Sensors **22**(4), 1430 (2022). https://doi.org/10.3390/s22041430
2. Fu, B., Bi, M., Umeda, S., et al.: Digital twin-based smart manufacturing: Dynamic line reconfiguration for disturbance handling. IEEE Transactions on Automation Science and Engineering **22**, 14892–14905 (2025). https://doi.org/10.1109/TASE.2025.3563320
3. Hespanha, J.P., Naghshtabrizi, P., Xu, Y.: A survey of recent results in networked control systems. Proceedings of the IEEE **95**(1), 138–162 (2007). https://doi.org/10.1109/JPROC.2006.887288
4. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital Twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine **51**(11), 1016–1022 (2018). https://doi.org/10.1016/j.ifacol.2018.08.474
5. Todorov, E., Erez, T., Tassa, Y.: MuJoCo: A physics engine for model-based control. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026–5033. IEEE (2012). https://doi.org/10.1109/IROS.2012.6386109
6. Yates, R.D., Sun, Y., Brown, D.R., III, Kaul, S.K., Modiano, E., Ulukus, S.: Age of information: An introduction and survey. IEEE Journal on Selected Areas in Communications **39**(5), 1183–1210 (2021). https://doi.org/10.1109/JSAC.2021.3065072
7. Namct123: Network-Aware Production Twin, software artifact (2026). Baseline commit 93d60ca; native candidate 20261103-native-physics. `https://github.com/Namct123/network-aware-production-twin`. Source and result snapshots: `docker/evidence/`.
8. MuJoCo: Python documentation, rollout module. `https://mujoco.readthedocs.io/en/stable/python.html#rollout`. Accessed 3 November 2026.